

Intrusion detection using mouse dynamics

ISSN 2047-4938

Received on 23rd May 2018

Revised 30th January 2019

Accepted on 20th February 2019

E-First on 25th March 2019

doi: 10.1049/iet-bmt.2018.5126

www.ietdl.org

Margit Antal¹ ✉, Elöd Egyed-Zsigmond²

¹Faculty of Technical and Human Sciences, Sapientia University, Calea Sighisoarei 1C, Tirgu Mures, Romania

²Université de Lyon, LIRIS UMR 5205 CNRS, Villeurbanne, France

✉ E-mail: manyi@ms.sapientia.ro

Abstract: Compared to other behavioural biometrics, mouse dynamics is a less explored area. General purpose data sets containing unrestricted mouse usage data are usually not available. The Balabit data set was released in 2016 for a data science competition which, despite the few users covered, can be considered the first adequate publicly available data set. This study presents a performance evaluation study on this data set for impostor detection. The existence of very short benchmark test sessions makes this data set challenging. Raw data were segmented into mouse actions, such as mouse move, point and click, and drag and drop, and then several features were extracted. Mouse data is not sensitive; therefore, it is possible to collect negative mouse dynamics data and to use two-class classifiers. Impostor detection performance was measured using decisions based on several actions. The best performance of 0.92 area under curve was obtained on the benchmark test sessions of the data set. Drag and drop mouse actions proved to be the best actions for impostor detection despite the fact that this type of action represented only 10% of the data set.

1 Introduction

Protecting accounts from unwanted access is becoming increasingly important. Online banking, online shopping or online examinations are part of our everyday routine. Several types of biometrics, such as fingerprint, iris, facial, and voice have been already used for this purpose. Unfortunately, they are, on the one hand, intrusive and on the other hand, they do not provide continuous protection. Behavioural biometric methods are not intrusive and provide continuous control.

Several behavioural biometrics have already been studied for intrusion detection. Computer keyboard usage is usually referenced as keystroke dynamics. The way a user types his own password can be used for static (entry point) authentication. These types of systems usually work with a high error rate (5.32% [1]); therefore it is accepted only as additional biometrics combined with more secure biometrics. Dynamic (continuous) authentication requires modelling the user's typing style. Both static and dynamic authentications require logging the users' typing, which may contain sensitive data.

Mouse usage is usually referenced as mouse dynamics. In contrast to keystroke dynamics, the study of users' mouse usage does not require sensitive data from the users. Moreover, in web-based environments, mouse usage is more frequent than keyboard usage [2]. This type of behavioural biometrics has already been used for static and dynamic authentications. One of the main limitations of some previous studies is that their results are not reproducible due to the unavailability of the data sets. Public data sets would allow us to compare different methods for a given problem in an objective way. This is particularly important in biometric research. Although Ahmed and Traore [3] published their data sets – because raw data is not included – their data sets are only suitable for performing certain types of measurements. Despite the fact that it has been written in several papers [4–6] that the result of their research is a public data set for mouse dynamics, in reality, such a data set did not exist until 2016 when a data set appeared thanks to the Budapest office of the Balabit company. This data set includes mouse pointer timing and positioning information; its purpose is to be a benchmark data set in the field of information technology security. This data set was first made

public during a data science competition [7]. Unfortunately, the results of the competition are not publicly available.

Another limitation of some previous studies is that experiments were conducted on data collected from a single application, usually specially designed for data collection [5, 6, 8]. In these applications, users were required to perform with the mouse some very specific tasks; therefore, these are not natural mouse usages.

The major contributions of our work are the following:

- We present a statistical analysis of the Balabit data set, which contains mouse data collected through general computer usage.
- We compare action-based and set of actions-based evaluations in order to determine the number of mouse actions required for high precision intrusion detection.
- We present measurements in order to determine which type of mouse action is the most user specific.

It is important to note that we took special care of the reproducibility of our measurements and described our experiments with great attention towards this objective. Our software and other resources of this research are available at <http://www.ms.sapientia.ro/~manyi/mousedynamics/>.

The rest of this paper is organised as follows. A literature review on mouse dynamics is presented in Section 2. This is followed by the description of the Balabit data set and its statistical analysis. Section 4 presents the methods used in this study, namely our feature extraction and the machine learning algorithm. Experimental results are presented in Section 5. This is followed by discussion and conclusions.

2 Background and related work

Behavioural biometrics is attracting increasing attention in security research. Keystroke, touch and mouse dynamics belong to this type of biometric. Keystroke dynamics is already used in on-line courses as a method for continuous authentication. However, keystroke data may contain sensitive data such as passwords and user names. In contrast, mouse dynamics data do not contain such sensitive data. Therefore, it is easy to collect and use this kind of information for intruder detection, e.g. in on-line banking systems. In mouse dynamics, the time taken to detect an intruder is

Table 1 Most important existing works on mouse dynamics

Paper	#Users	Data	Data availability	Features from	Main results	Note
Gamboa and Fred [8]	50	specific	no	stroke	EER: 0.2%	200 strokes
Ahmed and Traore [3]	22	general	yes	sequence of actions	EER: 2.46%	2000 actions
Nakkabi <i>et al.</i> [9]	48	general	yes	sequence of actions	FAR: 0%, FRR: 0.36%	2000 actions
Ahmed and Traore [10]	22	general	yes	sequence of actions	FAR: 0%, FRR: 0.03%	2000 actions
Shen <i>et al.</i> [4]	28	general	no	frequent behaviour pattern	FAR: 0.37%, FRR: 1.12%	3000 mouse
Shen <i>et al.</i> [5]	37	specific	no	8 consecutive mouse movements	FAR: 8.74%, FRR: 7.69%	11.8 s
Shen <i>et al.</i> [6]	58	specific	no	8 consecutive mouse movements	FAR: 8.81%, FRR: 11.63%	6.1 s
Zheng <i>et al.</i> [11]	30/1000	general/specific	no	action	EER: 1.3%	20 actions
Feher <i>et al.</i> [13]	25	general	no	action	EER: 8.53%	30 actions
Hinbarji <i>et al.</i> [16]	10	general	no	mouse curve	EER: 9.8%	100 curves
this paper	10	general	yes	action	EER: 2.6%	13 actions

dependent on the number of mouse events, and also on the types of mouse events (mouse click, mouse move, and drag-and-drop).

In this section, we review the most important works in terms of the following aspects: the type of data collection, data availability, research reproducibility, and the modelling unit features are extracted from. A distinction must be made regarding data collection types. Specific data is collected when we observe the usage of a certain application (web or desktop), while general data is generated when a user performs daily tasks on their operating system. From the point of view of reproducibility, the first requirement is the availability of the data used for experimentations. If the methods used are accurately described the research results can be reproduced. It is even better if the measurement software is also available. Reproducibility is a desirable property of good research. The data segment from which the features are extracted constitutes the modelling unit. The modelling unit may also depend on the type of data collection.

Gamboa and Fred [8] implemented a memory game as a web application and recorded the user interaction through the web page. They used mouse movements between successive clicks (stroke) as a modelling unit. This unit fits perfectly to the memory game the users had to play with during the data collection. The raw signal was smoothed, and then 63 spatial and temporal features were extracted. Feature selection was tuned for each user of the data set, and then a sequential classifier was applied. They reported 0.002 equal error rate (EER) for a user authentication experiment using sequences of 200 mouse movements. Their data set collected from 50 users is not public. In this study, we used a subset of their features.

The works of Ahmed, Traore, and Nakkabi [3, 9, 10] are presented in three consecutive papers based on the same data set. Two of these works [3, 10] present evaluations on the data set containing data from 22 users, while the third describes the measurement results on an extended data set (48 users). This extended data set is publicly available. The data set does not include raw data, but segmented and processed data. The data are available, the used methods are accurately described, therefore the research is reproducible. Raw data were segmented into mouse actions. Three types of mouse actions were considered: PC – point and click: mouse movement ending in a mouse click; MM – general mouse movement; DD – drag and drop. Histogram-based features were extracted from sequences of mouse actions. In this study, we have also used these three types of actions.

General mouse data were collected from 28 subjects by Shen *et al.* [4]. Raw mouse data were segmented into mouse actions. These mouse actions were encoded into mouse operations. A mouse operation contains relevant information about the application in which the action occurred. Frequent behaviour patterns were identified, and then features were extracted from these patterns. They reported that the features extracted from frequent-behaviour segments were more stable than holistic features used in previous papers. Very good performances were reported in a continuous user authentication framework using 30 min of mouse data.

Shen *et al.* [5] investigated static authentication using mouse dynamics. Their data set containing specific data was collected from 37 subjects, where each subject executed a fixed mouse

operation task (eight mouse movements) in a controlled environment. All the subjects used the same computer for data collection. The data set contains 5500 samples from the 37 subjects. The modelling unit was the mouse movement, from which 13 features were extracted, resulting in a 104-dimensional feature vector for the mouse operation task. They reported 8.74% false acceptance rate (FAR) and 7.69% false rejection rate (FRR) for a mouse operation task (11.8 s on average). In new research, Shen *et al.* expanded their data set from 37 to 58 subjects [6]. They presented a performance evaluation for a static authentication system using 17 different anomaly detectors. The original data sets used in these three papers [4–6] are not publicly available.

Zheng *et al.* [11, 12] collected two data sets. Their first data set contains general mouse usage data from 30 subjects. The second data set was collected from an online forum and contains the data of 1000 users. The raw data were segmented into the point and click actions. Angle-based features were extracted from the mouse actions. Besides these angle-based metrics, statistical features were extracted from blocks of actions. Their best performance for user authentication was 1.3% EER using a block of 20 mouse actions. These results cannot be reproduced due to the unavailability of the data sets.

Feher *et al.* [13] collected general mouse usage data from 25 subjects. Raw data were segmented into mouse actions. It is important to note that they defined seven types of mouse actions and extracted a different number of features from each type of action. The best performance in terms of EER was 8.53% using 30 actions for predictions. This data set is not publicly available.

Chuda and Kratky [14] studied user authentication in the web browsing process. They collected specific mouse usage data from 28 users and reported user authentication accuracy of 87.5% by using 20 features extracted from mouse data. Unfortunately, the authentication protocol, namely the amount of data used for authentication is not clearly described. Although the data set is not a general purpose one, it is publicly available. In a later paper, Chuda *et al.* [15] reported user identification in the web browsing process using only mouse clicks. They reported 96% user identification accuracy on a data set containing data from 20 users by using features extracted from 100 clicks.

Hinbarji *et al.* [16] collected a general mouse usage data set from 10 subjects. They used mouse movement curves (actions) as the basic modelling unit for user authentication. These mouse curves were characterised by nine features. A binary classifier using a neural network was created for each user of the data set. They obtained 9.8% EER by using 100 mouse curves (~5.6 min of mouse data) for user authentication. This data set is not publicly available.

In Table 1, we have summarised the important characteristics of the most important existing works on user authentication using mouse dynamics. For comparison, the last line of the table contains our own results.

Mouse dynamics was used not only for authentication but also for individual performance monitoring and stress detection. Carneiro and Novais [17] measured the individual performance of the users by their mouse dynamics data. In another paper, Carneiro

Table 2 Number of training and test sessions

User	Training	Test positive	Test negative
1	7	36	37
2	7	23	43
3	7	56	49
4	6	45	70
5	6	68	38
6	7	30	20
7	7	37	22
8	6	38	33
9	7	43	20
10	5	35	73
total	65	411	405

record timestamp	client timestamp	button	state	x	y	Result	
0.000	0.000	Left	Pressed	678	156	DD	DD action segment
0.102	0.109	NoButton	Drag	712	171	DD	
0.214	0.218	NoButton	Drag	805	212	DD	
0.326	0.218	NoButton	Drag	823	219	DD	
0.326	0.327	NoButton	Drag	851	227	DD	
0.406	0.405	NoButton	Drag	859	228	DD	
0.407	0.405	Left	Released	859	228	DD	PC action segment
0.507	0.515	NoButton	Move	853	228	PC	
0.622	0.624	NoButton	Move	777	263	PC	
0.726	0.624	NoButton	Move	748	284	PC	
0.726	0.733	NoButton	Move	554	381	PC	
0.844	0.733	NoButton	Move	544	387	PC	
0.844	0.858	NoButton	Move	541	399	PC	
1.088	0.951	NoButton	Move	530	438	PC	
1.088	1.061	NoButton	Move	530	460	PC	
1.166	1.061	NoButton	Move	530	464	PC	
1.166	1.170	NoButton	Move	531	471	PC	
1.394	1.404	Left	Pressed	531	471	PC	
1.454	1.451	Left	Released	531	471	PC	

Fig. 1 Mouse data segmentation into actions

et al. studied stress detection during online examinations using mouse movements' dynamics [18].

Automatically inferring the gaze position from mouse movements is another research field. Guo and Agichtein [19] reported 77% accuracy for predicting gaze position from mouse movements in web searching environment.

3 Balabit mouse challenge data set

3.1 Data set description

The Balabit mouse challenge data set was published in August 2016. It was used in a mouse dynamics data science challenge: 'The goal of the challenge was to protect a set of users from the unauthorised usage of their accounts by learning the characteristics of how they use their mice' [7].

The data set contains two folders, one for the training files and one for the test files. Data were collected from ten users working over remote desktop clients connected to remote servers. The original user identifiers in the data set are 7, 9, 12, 15, 16, 20, 21, 23, 29, and 35. We labelled these identifiers with numbers from 1 to 10. Both training and test data are presented as sessions, however, the test sessions are much shorter than the training sessions. The test part of the data set contains both positive (legal) and negative (illegal) sessions. The number of training and test sessions is shown in Table 2.

3.1.1 Raw data: A session file contains a set of lines and each line represents a recorded mouse event. Each recorded mouse event contains six fields: (*rtime*, *ctime*, *button*, *state*, *x*, *y*). The *rtime* is the elapsed time in seconds since the start of the session recorded by the network monitoring device. The *ctime* is also the elapsed time but as recorded by the client computer. The *button* field

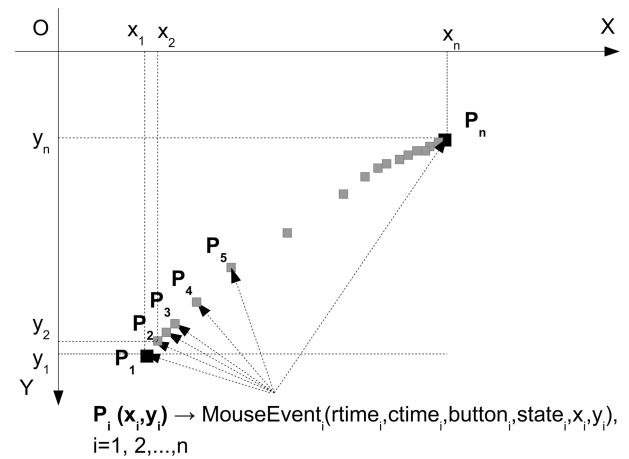


Fig. 2 Mouse action: a sequence of consecutive mouse events which describes the movement of the mouse between two screen positions

represents the current condition of the mouse buttons, and the field *state* contains additional information about the button. The *x* and *y* fields are the coordinates of the cursor on the screen (see Fig. 1).

Before segmenting the raw data into meaningful units it had to be cleaned. First, the duplicated entries were filtered out, then the instances where the mouse was moved outside the remote desktop window (big *x*, *y* coordinates) were replaced by previous coordinates.

The data set also contains recorded mouse scroll events. These events were generated by the scroll button of the mouse. In order to decide whether or not to use these data, we counted the sessions containing this type of event. Since 42% of the test sessions do not contain scroll events, it was decided not to use this type of raw data.

3.2 Segmentation

A mouse action is a sequence of consecutive mouse events, which describes the movement of the mouse between two screen positions. Since each event contains a screen position, a mouse action which consists of *n* events is depicted as a sequence of *n* points: $\{P_1, P_2, \dots, P_n\}$ (see Fig. 2).

We segmented each session's data into three types of mouse actions. These action types were defined by Ahmed and Traore [3] as: $\{MM, PC, DD\}$. *MM* denotes a general mouse movement action. This type of action simply moves the mouse pointer between two points of the screen. *PC* denotes a point click action. This is similar to the mouse movement action, but it ends by a mouse click. *DD* denotes a drag and drop action, which always starts with a left mouse button pressed event, followed by a sequence of mouse drag events, and ends by a left mouse button released event.

Raw data were split into actions in two steps. In the first step, we cut the raw data into segments ending in mouse released events. When the previous event of a mouse released event is a mouse pressed event then we have a segment ending in a PC type action, otherwise, we have a segment ending in a DD type action (see Fig. 1). These segments may contain zero or several MM type actions before the ending in a PC or DD type action. In the second step, the MM actions are isolated using a threshold for the time field. We always divide the segment into two actions when the time difference between two consecutive events is higher than a threshold. We used 10 s as the time threshold. Finally, short actions having less than four events were ignored (spline interpolation requires at least four points).

An example of raw data segmentation into actions can be seen in Fig. 1. In this figure, two complete segments are shown. Both segments contain only one action.

3.3 Data set statistics

Table 3 presents the action type statistics for the training and test parts of the data set. It can be seen that the distribution of the three

types of actions is similar in the training and the test parts of the data set. PC actions represent about 70% of the data, MM actions about 20% and the remaining 10% are the DD actions. A training session contains 937 actions on average, at least 237 and at most 2488. Test sessions are much shorter. They contain 50 actions on average, at least four and at most 175. This means that even in session-based evaluations for the test part the decision has to be taken based on only four actions.

In order to see the distribution of action types at the user level, we computed action type histograms for each user of the data set (see Fig. 3). For this diagram, we used only the training part of the data set. We can see that action type patterns of the users are very similar, despite the fact that during data collection, users performed their individual daily activities.

4 Proposed user model

This section presents the features extracted from mouse actions as well as the machine learning algorithms used for evaluations. The architecture of our user model creation process is shown in Fig. 4.

4.1 Feature engineering

We present here the features extracted from individual mouse actions. Similar approaches (using different features) were already studied in [11–13]. Gamboa and Fred [8] extracted features from action-like units, although they used the term stroke instead of action. We have already defined a mouse action as a sequence of consecutive mouse events which describes the movement of the mouse between two screen positions. Even though each mouse event contains six fields (corresponding to a line in the raw data), only three of these are used in the feature extraction: the x and y fields, and the client time stamp $ctime$. The other fields were used in the segmentation step. Therefore, each action can be represented by a sequence of triplets (x_i, y_i, t_i) , $i = 1, \dots, n$, where n is the number of mouse events in the action.

Based on the x and y sequences, we computed Θ , which is the angle of the path tangent with the x -axis calculated using (1):

$$\Theta_i = \text{atan2}\left(\frac{\delta y_i}{\delta x_i}\right), i = 2, \dots, n; \Theta_1 = 0, \quad (1)$$

where $\delta y_i = y_i - y_{i-1}$ and $\delta x_i = x_i - x_{i-1}$. The atan2 trigonometric function is a common variation of the standard arctangent function, which produces results in the range $(-\pi, \pi]$. The following six time series: v_x – horizontal velocity, v_y – vertical velocity, v – velocity, a – acceleration, j – jerk, and ω – angular velocity defined by (2) and (3) were computed based on $x(t)$, $y(t)$, and $\Theta(t)$ series. These six time series were proposed by Gamboa and Fred [8]:

$$v_{xi} = \frac{\delta x_i}{\delta t_i}, v_{yi} = \frac{\delta y_i}{\delta t_i}, v_i = \sqrt{v_{xi}^2 + v_{yi}^2}, i = 2, \dots, n, \quad (2)$$

where $\delta t_i = t_i - t_{i-1}$, $\delta t_1 = 0$, $v_{x1} = 0$, $v_{y1} = 0$, $v_1 = 0$, and

$$a_i = \frac{\delta v_i}{\delta t_i}, j_i = \frac{\delta a_i}{\delta t_i}, \omega_i = \frac{\delta \Theta_i}{\delta t_i}, i = 2, \dots, n, \quad (3)$$

where $a_1 = 0$, $j_1 = 0$, and $\omega_1 = 0$.

The curvature time series c is defined as the ratio between the angle change and the travelled distance (4)

$$c_1 = 1, \quad c_i = \frac{\delta \Theta_i}{\delta s_i}, i = 2, \dots, n, \quad (4)$$

where $s_i = \sum_{k=1}^i \sqrt{\delta x_k^2 + \delta y_k^2}$ is the length of the trajectory from the starting point of the action to the i th point, and $\delta s_i = s_i - s_{i-1}$, $i = 2, \dots, n$, and $s_1 = 0$. Critical points are the low curvature points ($c_i < TH_c$, where $TH_c = 0.0005$ was determined empirically). We used the number of critical points as a feature.

Mean, standard deviation, and maximal values of the defined six time series were used as features. In addition, we used the following features: the type of action, the length of the trajectory (s_n), the length of the segment between the two endpoints, and the time needed to perform the action. Direction feature is the direction of the end to end line. To reduce the possible direction values, we used the eight main directions defined by Ahmed and Traore [3] (see Fig. 5). Straightness was computed as the ratio between the segment between the two endpoints of an action and the length of the trajectory. Other used features were: the number of points, which means the number of mouse events contained in an action, the sum of angles, and the largest deviation, which is the largest distance between the points of the trajectory and the segment between the two endpoints. Another used feature is the duration of the first segment of the action with positive acceleration a_beg_time ($\sum_{i=2}^k \delta t_i$, where $k+1$ is the leftmost index when $a_{k+1} \leq 0$) were also used as features. Table 4 shows all the features extracted from a mouse action.

The distribution of different features in the training and test parts of the data set are shown in Fig. 6. It can be seen that the distribution of features in the training and the test parts are similar. The travelled distance histogram shows the frequency of actions within different distance ranges. Usually, users performed actions with short travelled distances more frequently than with long ones. In the straightness histogram, we can see that there are many relatively straight actions in both the test and the training parts of the data set. This is also confirmed by the histogram of the curvature of the actions. The velocity histogram shows that users prefer low-speed movements. Angular velocity histograms reveal the tendency of the users making mostly straight mouse movements. The histogram of the distance between the two endpoints is similar to that of the travelled distance. Users make mouse movements having a low number of critical points. This is consistent with straightness, curvature and largest deviation histograms.

4.2 User profiles

It is known that in some types of behavioural biometrics, binary or two-class classifiers perform better than one-class classifiers [20]. Using information about other users' behaviour results in better user profiles. However, it is not always possible to collect negative data (e.g. keystroke dynamics, password typing rhythm). Fortunately, in mouse dynamics, this is possible, therefore, we used binary classifiers.

For each user, we created a binary classifier, which is responsible for deciding whether some amount of mouse data belong or not to a given user. Several classifiers integrated into the Weka Machine learning toolkit [21] were tried and Random forest algorithm [22] proved to be the best. This algorithm performs well even in the case of correlated features. Therefore, we trained a separate Random forest classifier for each user. These classifiers were trained by using balanced training sets.

Training sets contain two classes: the genuine (positive) and the impostor (negative). For each user, we selected a user-specific data set having an equal amount of genuine and impostor mouse actions in order to avoid bias. A user having N_i genuine mouse actions would have in its user-specific data set these N_i genuine actions and N_i impostor actions. Impostor actions were selected from the other users. Having a total number of n users, for each user-specific data set, we have one genuine user (positive data samples) and $n-1$ impostor users (negative data samples). Consequently, $N_i/(n-1)$ impostor actions were selected from each impostor user.

After the training stage, the created classifiers were evaluated. Two types of evaluations were used: action-based and set of actions-based. In the action-based evaluation, the classifier output is a score, which represents the probability of the action belonging to the positive class. However, in the case of a set of actions based evaluation, we need the decision to be taken based on all the actions in the set. In the following, we explain the score computation for this case. Let $A = \{a_1, a_2, \dots, a_k\}$ be a set of actions. For each action, we computed the probability of belonging

Table 3 Types of actions in the training and test parts of the data set

	MM	PC	DD
training	12,828 (21.06%)	41,610 (68.32%)	6467 (10.62%)
test	8597 (20.90%)	28,677 (69.70%)	3867 (9.40%)

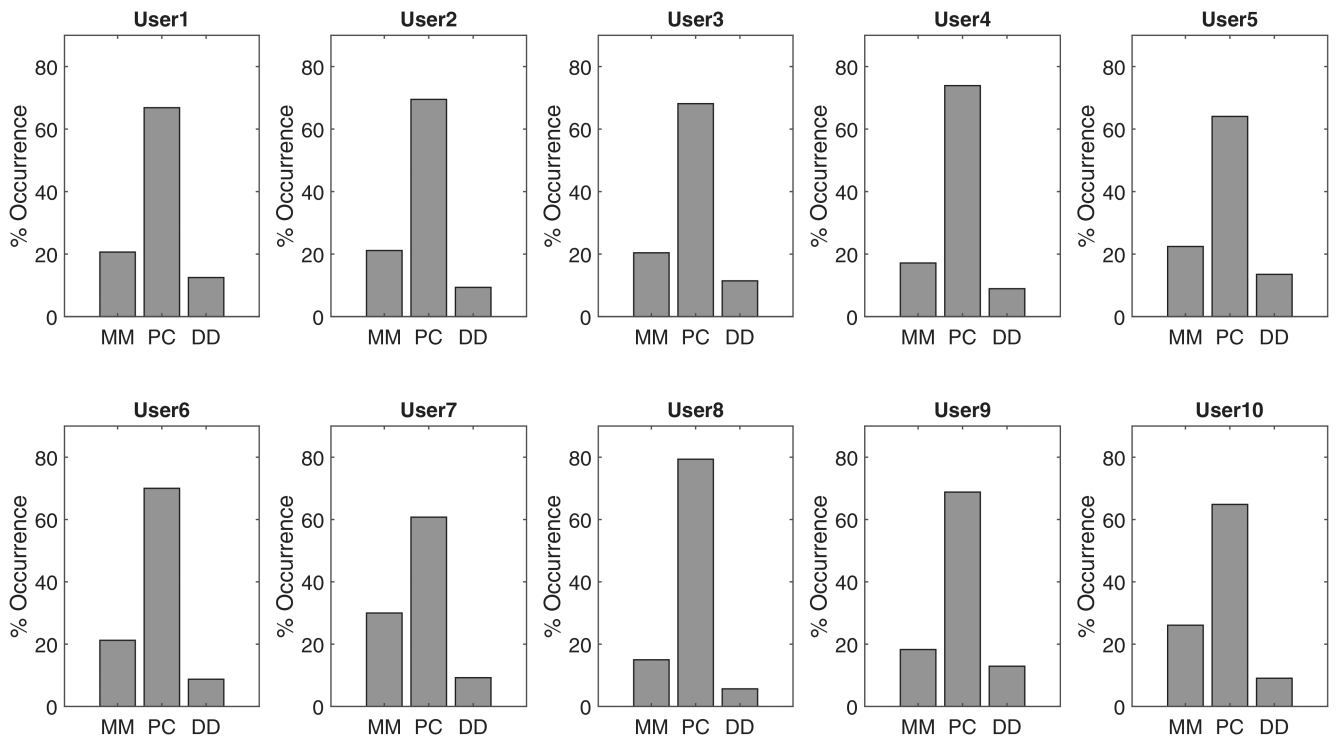


Fig. 3 Histograms of the types of actions for all the ten users using the training part of the data set

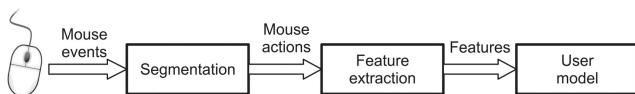


Fig. 4 User model creation process

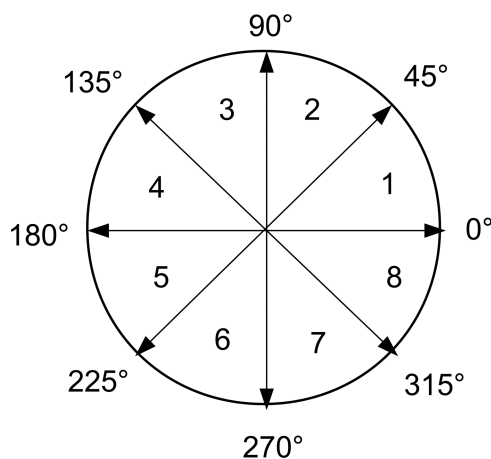


Fig. 5 Eight directions. Angles between 0° and 45° fall into the direction 1

to the positive class. (Specifically, the `distributionForInstance` method of the Weka classifier was used, which predicts the class memberships for a given instance.) This resulted in a set of k probabilities: $\{p_1, p_2, \dots, p_k\}$, then we computed a single score for the set of actions as the average of the actions probabilities

$$P(A) = \frac{1}{k} \sum_{i=1}^k p_i. \quad (5)$$

We used $P(A)$ as the score for the set of actions A . The performance of the classifier depends on the number of actions that are used to estimate the authenticity of a user. The influence of the number of actions on the performance of the classifier was analysed for two different evaluation scenarios in Sections 5.3.3 and 5.4.2.

5 Experimental results

5.1 Evaluation metrics

We report evaluation results in terms of classification accuracy (ACC), area under curve (AUC), false negative rates (FNRs), false positive rates (FPRs), and equal error rates (EERs). Besides these values, we report the performance of our detection system by receiver operating characteristic (ROC) curves [23]. ROC curves are especially meaningful for biometrics because they show the performance of the biometric system on several operating points corresponding to different decisional thresholds.

Accuracy is defined as the percentage of correctly classified instances. In the case of ROC curves, first, we computed a positive score list using the positive test data and a negative score list using the negative test data. Then, based on these two lists the FPRs and FNRs were computed for different threshold values (threshold $\in [0, 1]$). FPR is the ratio of the number of negative samples accepted as positive divided by the total number of negative samples (FPR). FNR is the ratio of the number of positive samples rejected divided by the total number of positive samples (FNR). EER values express the error rate for a binary classifier detection threshold for which the FPR and the FNR are equal. The AUC value is computed using the ROC curve, which plots FPRs against true positive rates: it is the area under the ROC curve. 'The AUC of a classifier is equivalent to the probability that the classifier will rank a randomly chosen positive instance higher than a randomly chosen negative instance' [23]. AUC is often used to compare classifiers.

Table 4 Features extracted from a single mouse action

Name	Description	#features
v_x	mean, std, min, max	4
v_y	mean, std, min, max	4
v	mean, std, min, max	4
a	mean, std, min, max	4
j	mean, std, min, max	4
ω	mean, std, min, max	4
c	mean, std, min, max	4
type	MM, PC, DD	1
elapsed time	$t_n - t_1$	1
trajectory length	s_n	1
dist_end_to_end	$ P_1 P_n $	1
direction	see Fig. 5	1
straightness	$\frac{ P_1 P_n }{s_n}$	1
num_points	n	1
sum_of_angles	$\sum_{i=1}^n \Theta_i$	1
largest_deviation	$\max_i \{d(P_i, P_1 P_n)\}$	1
num_critical_points	$\# \{c_i c_i < TH_c\}$	1
a_beg_time	accel. time at the beginning	1
total		39

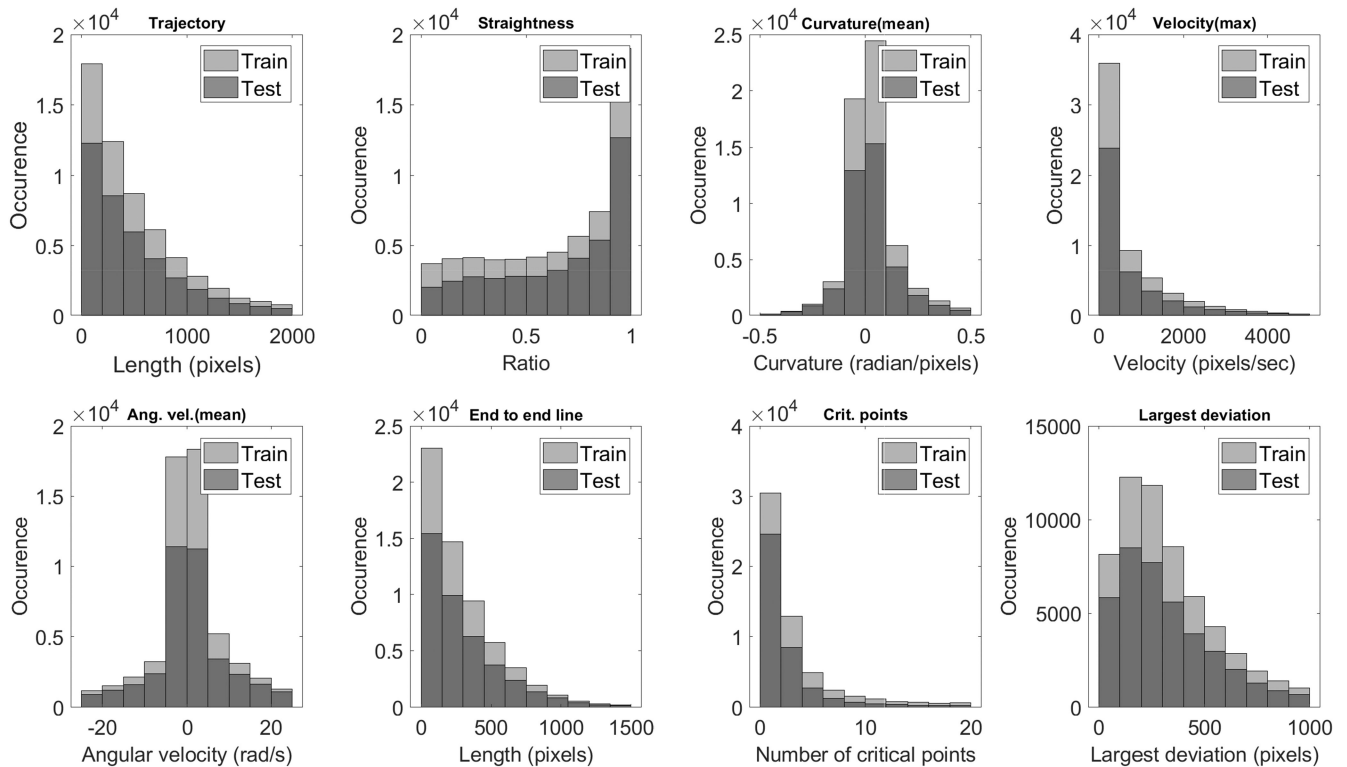


Fig. 6 Distribution of different features in the training and the test parts of the data set

All the evaluations were performed with a Java application using Weka Machine Learning toolkit (version 3.8).

5.2 Evaluation scenarios

The Balabit data set has two parts: a training and a test part. We are going to present two evaluation scenarios: (A) both the training and the testing were performed using exclusively the training part of the data, (B) the training had been performed on the training part of the data set and the testing on the test part of the data set.

In both scenarios, we evaluated the binary classifiers using decisions: (i) based on a single action, (ii) based on a set of actions (or sequence of actions). Evaluation based on a set of actions is also referred to as periodic authentication [24] because the users

can always perform a fixed number of actions before the system checks their identity.

5.3 Scenario A: Balabit data set – using only the training part

In this subsection, we describe the details of experiments based on the training part of the Balabit data set. This part of the data set contains 60,905 actions from 65 sessions.

5.3.1 Action-based evaluation: A binary classifier was built from the user-specific data set selected as explained earlier. These data sets were evaluated using Random forest classifier and ten-fold cross-validation. Table 5 reports detailed results for each of the

Table 5 Scenario A: action-based evaluation

User	ACC%	AUC	FNR	FPR
1	93.27	0.9725	0.0138	0.1208
2	97.41	0.9975	0.0132	0.0386
3	77.17	0.8560	0.1835	0.2736
4	77.43	0.8621	0.1642	0.2873
5	72.69	0.8132	0.1648	0.3814
6	78.54	0.8731	0.1759	0.2532
7	76.65	0.8462	0.1739	0.2931
8	75.74	0.8414	0.1960	0.2892
9	80.50	0.8954	0.2017	0.1881
10	72.01	0.8010	0.1967	0.3632
avg	80.17	0.8761	0.1484	0.2488
std.	8.45	0.0637	0.0723	0.1057

Table 6 Scenario A: using only one type of action

User	MM		PC		DD	
	ACC%	AUC	ACC%	AUC	ACC%	AUC
1	91.50	0.9576	93.69	0.9764	89.86	0.9569
2	95.62	0.9953	97.65	0.9984	94.84	0.9902
3	75.00	0.8465	76.85	0.8504	84.28	0.9240
4	78.58	0.8599	76.29	0.8513	83.77	0.9230
5	72.20	0.8041	72.26	0.8001	78.00	0.8644
6	75.71	0.8334	78.09	0.8638	81.89	0.9071
7	75.53	0.8328	75.82	0.8413	75.94	0.8386
8	73.97	0.8152	74.54	0.8302	79.32	0.8725
9	78.05	0.8679	79.61	0.8889	85.92	0.9428
10	72.23	0.7928	71.71	0.7970	73.59	0.8290
avg.	78.84	0.8605	79.65	0.8698	82.74	0.9048
std.	8.09	0.0660	8.83	0.0679	6.46	0.0527

Table 7 Scenario A: set of actions-based evaluations

#Actions	AUC	EER
1	0.8806	0.2128
2	0.9490	0.1274
3	0.9763	0.0808
4	0.9883	0.0545
5	0.9940	0.0375
6	0.9968	0.0269
7	0.9982	0.0186
8	0.9991	0.0136
9	0.9995	0.0096
10	0.9997	0.0069
11	0.9998	0.0051
12	0.9999	0.0039
13	1.0000	0.0026
14	1.0000	0.0018
15	1.0000	0.0017
16	1.0000	0.0016
17	1.0000	0.0012
18	1.0000	0.0014
19	1.0000	0.0007
20	1.0000	0.0004

ten users of the data set. Besides the user-specific evaluation metrics, the mean and standard deviation are also reported. Using ten-fold cross-validation 80.17% average accuracy and 0.87 average AUC was obtained. We should note that each classifier was evaluated separately. This means that user-specific metrics were computed based on user-specific decision thresholds. We report the average and standard deviation of these values. It is important to note that high accuracies (over 93%) and also high

AUC values (over 0.97) were obtained for two users (1 and 2), and lower accuracies (between 72 and 80%) and also lower AUC values (between 0.80 and 0.89) for the other eight users. Except for a few users (1, 2 and 9), the FPRs are much higher than the FNRs. This means that using only one mouse action a lot of impostors are accepted as they were genuine users.

5.3.2 Which type of action is the most user specific?: The training data were separated into three sets, each set containing only MM, PC, or DD actions, respectively. In order to see which of the three types of actions is the most user specific, we repeated the action-based evaluation three times using the corresponding training set. Detailed results are shown in Table 6. It can be seen that the results are very similar for MM and PC actions. This is not a surprising result because the only difference between the two types of actions is that PC actions contain a mouse click event also. Even though DD type actions are 10% of the data set, if only this type of action is used for both model building and testing, the results are 3% better than for using only PC or only MM type actions.

5.3.3 Set of actions-based evaluation: In this case, scores were computed for sets of actions varying the number of actions in a set. In this case, we report the performance of the whole system. This means that both AUC and EER were computed based on a global threshold, which was computed using a common positive and negative score lists for all users of the data set. Table 7 shows the AUC and EER values as a function of a number of actions. It can be seen that after 13 actions the AUC became 1, however for an EER of 0.0004 (0.04%), 20 actions were necessary.

5.3.4 Informativeness of the features: Informativeness of the features has also been evaluated using the GainRatioAttributeEval evaluator from the Weka Machine Learning toolkit (version 3.8). This evaluator computes the gain ratio with respect to the class for

Table 8 Informativeness of the features

Ranking	Feature	Gain ratio	Ranking	Feature	Gain ratio	Ranking	Feature	Gain ratio
1	a_beg_time	0.13422	14	max_curv	0.06654	27	min_v	0.02255
2	num_critical_points	0.12266	15	min_vy	0.06428	28	sd_vy	0.02223
3	min_jerk	0.09648	16	min_a	0.06339	29	sd_vx	0.02141
4	num_points	0.09458	17	max_vx	0.06182	30	mean_omega	0.02093
5	min_omega	0.09401	18	min_vx	0.06071	31	straightness	0.02002
6	max_omega	0.09292	19	max_a	0.0522	32	trajectory_length	0.01982
7	mean_jerk	0.08626	20	sd_a	0.05038	33	dist_end_to_end	0.01751
8	sd_jerk	0.08089	21	sd_curv	0.03538	34	mean_v	0.01732
9	min_curv	0.07969	22	elapsed_time	0.03416	35	mean_curv	0.01396
10	mean_a	0.0774	23	max_v	0.03385	36	mean_vx	0.01154
11	max_jerk	0.0754	24	sd_omega	0.03367	37	mean_vy	0.01121
12	sum_of_angles	0.0743	25	largest_deviation	0.0295	38	type_of_action	0.00996
13	max_vy	0.06765	26	sd_v	0.02501	39	direction	0.00291

Table 9 Scenario B: action-based evaluation

User	ACC%	AUC	FNR	FPR
1	98.63	0.9987	0.0128	0.0146
2	99.16	0.9997	0.0170	0.0038
3	68.61	0.7666	0.2137	0.4167
4	53.20	0.6293	0.2729	0.5777
5	63.54	0.6226	0.1854	0.6917
6	82.68	0.9192	0.1522	0.2053
7	66.54	0.6676	0.1963	0.5906
8	65.64	0.7270	0.2298	0.4434
9	70.96	0.7389	0.2260	0.4511
10	53.94	0.6761	0.1708	0.5976
avg.	72.29	0.7746	0.1677	0.3992
std.	16.30	0.1456	0.0873	0.2450

Table 10 Scenario B: set of actions-based evaluations

#Actions	AUC	EER
1	0.7864	0.3024
2	0.8214	0.2684
3	0.8392	0.2526
4	0.8512	0.2415
5	0.8600	0.2342
6	0.8667	0.2277
7	0.8727	0.2271
8	0.8759	0.2168
9	0.8792	0.2114
10	0.8838	0.2086
11	0.8857	0.2051
12	0.8863	0.2061
13	0.8914	0.1953
14	0.8908	0.1951
15	0.8932	0.1924
16	0.8943	0.1944
17	0.8963	0.1914
18	0.8964	0.1952
19	0.8983	0.1897
20	0.8994	0.1880

each feature. Larger gain ratios indicate better features. Features ranking is shown in Table 8. It can be seen that the most informative feature was the acceleration at the beginning of the mouse action followed by the number of critical points. Jerk related features were also found to be more informative than velocity related features. The least informative feature was the direction of the action.

5.4 Scenario B: Balabit data set – using both training and test parts

In this subsection, we describe the details of the evaluations based on the whole Balabit data set. The main difference compared to scenario A is that, in this section, the training part of the data set was used only for building the classifiers, and all the testings were performed exclusively on the test part of the data set.

5.4.1 Action-based evaluation: The binary classifiers were built similar to scenario A, except that we used all the data in the training part for training the classifiers. Having the trained classifiers, we used the test data (both positive and negative sessions) and evaluated the classifiers. The detailed results are reported in Table 9. Compared to the same type of evaluation used in scenario A (see Table 5), we observed a decrease of 10% in terms of AUC and 8% in terms of accuracy. In terms of errors, in the case of FNR, we observed a small increase, but the FPR almost doubled.

5.4.2 Set of actions-based evaluation: In this case, scores were computed for sets of actions varying the number of actions in a set. We report the AUC and EER values as a function of the number of actions. We note that test sessions contain a varying number of actions (between 4 and 175, on average 50). Let us denote m the number of actions contained in a test session, and n the number of actions contained in a set of actions. Then, for this session, we computed m/n scores, one for each n -length set of action. Sessions containing less than n actions were ignored. Table 10 shows the AUC and EER values as a function of a number of actions. It is worth comparing Tables 7–10. We can observe the same tendency as for action-based evaluation. AUC values are lower with 10% compared to the same type of evaluation in scenario A. The best AUC is 0.89 and the corresponding EER is 0.18.

5.4.3 Session-based evaluation: This evaluation is similar to the set of actions-based one. In this case, we used also a set of actions for decision: all the actions belonging to a test session. For each session, we computed exactly one score. The number of scores obtained is equal to the number of test sessions (816).

Scores were computed for all positive and negative test sessions, which resulted in a positive and negative score lists. The distribution of these scores and its corresponding ROC curve is shown in Fig. 7. For comparison, we included the ROC curve obtained for action-based evaluation and the corresponding scores' distribution too. We observe an increase of 14% in terms of AUC when using session-based evaluation.

5.4.4 Effect of signal smoothing: In order to smooth the unevenly (event-based) sampled signal, we interpolated the mouse movements to temporally homogeneous samples (the smoothed curve was re-sampled with a frequency of 20 Hz). Both linear and spline interpolation were applied to the signal. We report results for

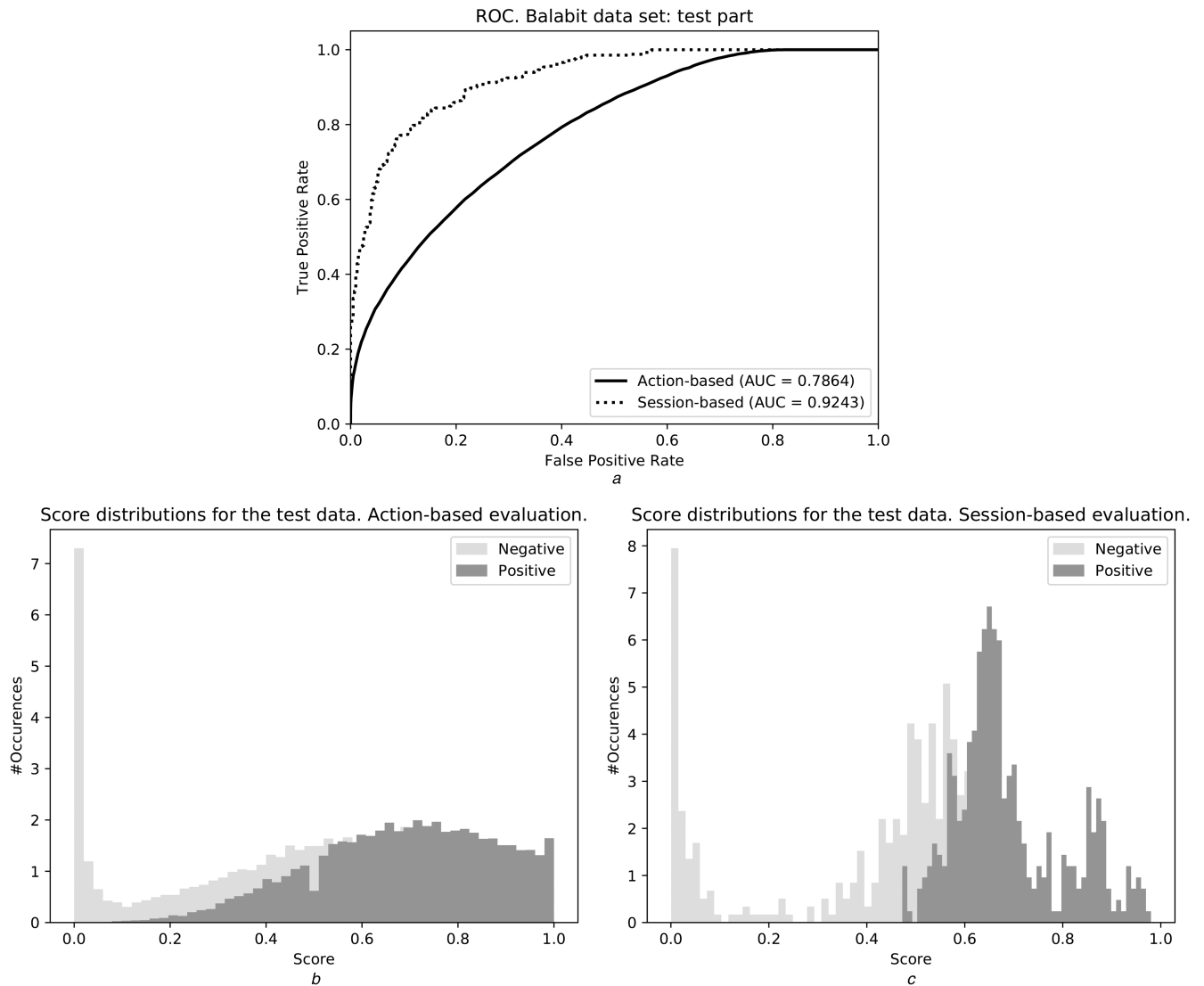


Fig. 7 ROC curves. Action-based versus session-based evaluations
(a) ROC, (b) Score distribution – action-based evaluation, (c) Score distribution – session-based evaluation

scenario B, session-based evaluation. The results are depicted in Fig. 8. The spline interpolation did not change the result significantly, but the linear interpolation resulted in a 2% decrease in the AUC value.

6 Discussion

Gamboa and Fred [8] demonstrated experimentally that mouse data collected through a memory game (all the users working with the same application) can be used for user authentication by achieving 0.2% EER for 200 mouse strokes (a kind of mouse action). Ahmed and Traore [3] extended Gamboa and Fred's work by collecting mouse data resulting from a general computer usage scenario (all the users performing their daily work). They obtained 2.46% EER by using 2000 actions for authentication. Despite the good results, the high number of actions required to verify a user's identity makes their method impractical for access control systems.

Compared to these two works our result of 0.04% EER obtained for 20 actions is very good. We should note that this very good result was obtained only for scenario A (evaluation based on the training part of the data set). A more modest result of 18.80% EER (20 actions) was obtained for the test part of the data set.

Like Feher *et al.* [13], we evaluated methods based on both single actions and aggregation of actions (set of actions). They reported the best EER for using 30 actions for authentication. Our findings are similar for the Balabit data set: using more than 15 actions, the EER and AUC values did not improve significantly.

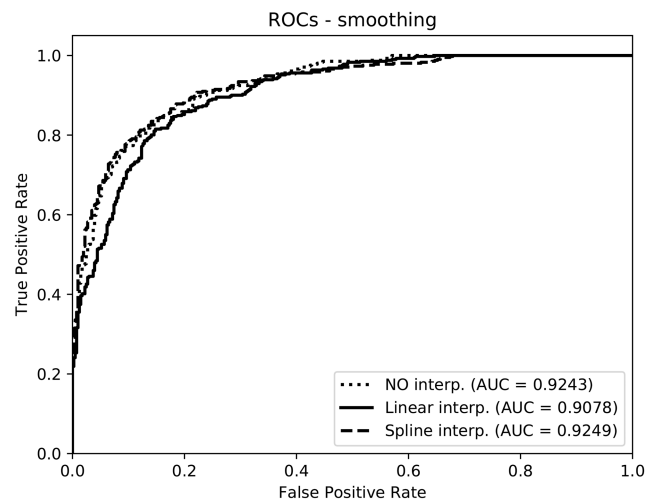


Fig. 8 Effect of signal smoothing. Evaluation: session-based

We reported our findings related to the Balabit data set, which is quite a challenging one due to the short test sessions. Besides the good detection results, we were particularly attentive to the reproducibility of our measurements. The results may not be scalable to a larger system, because it contains data only from ten users. There is still a need for a larger mouse dynamics data set.

7 Conclusions

In this study, we have presented an intruder detection study using the Balabit mouse dynamics data set. This data set was specially developed for intruder detection. We especially took care of the reproducibility of our work. Both the data set and the software are publicly available.

The data set was evaluated using two scenarios: (A) using only the training part of the data set and (B) using the whole data set. Scenario B proved to be quite challenging due to the short benchmark test sessions. In both scenarios, we used binary classifiers. These binary classifiers were evaluated based on single action and set of actions.

We have seen that one action does not contain enough information about the user mouse usage behaviour, therefore more actions are required for an accurate decision. Scenario A resulted in almost perfect classification using 20 actions for detection (AUC: 1, EER: 0.04%), however in scenario B more modest results were obtained for the same number of actions (AUC: 0.89, EER: 18.80%). A slight improvement was obtained by session-based evaluation (0.92 AUC). In this case, the benchmark test sessions contained 50 actions on average, at least 4 and at most 175.

Action-specific classifiers were trained using only one type of mouse action. Despite the fact that drag and drop actions constitute only 10% of the data set, these proved to be the most user-specific actions.

Both linear and spline interpolation were applied to the unevenly sampled mouse data. The interpolation did not improve the intruder detection system accuracy.

Our findings suggest that mouse dynamics can be considered in an intruder detection system along with other behavioural biometrics. Nevertheless, one should have enough test data in order to perform the detection with good accuracy. We chose the Balabit data set because to the best of our knowledge it is the only available raw mouse usage data set. We are actively working on a collection protocol and system in order to be able to create our own mouse dynamics data set for intruder detection.

8 References

- [1] Morales, A., Fierrez, J., Tolosana, R., *et al.*: 'Keystroke biometrics ongoing competition', *IEEE Access*, 2016, **4**, pp. 7736–7746
- [2] Kratky, P., Chuda, D.: 'Recognition of web users with the aid of biometric user model', *J. Intell. Inf. Syst.*, 2018, **4**, pp. 1–26
- [3] Ahmed, A.A.E., Traore, I.: 'A new biometric technology based on mouse dynamics', *IEEE Trans. Dependable Secur. Comput.*, 2007, **4**, (3), pp. 165–179
- [4] Shen, C., Cai, Z., Guan, X.: 'Continuous authentication for mouse dynamics: a pattern-growth approach'. Proc. 2012 42nd Annual IEEE/IFIP Int. Conf. on Dependable Systems and Networks (DSN), Washington, DC, USA, IEEE Computer Society, 2012, pp. 1–12
- [5] Shen, C., Cai, Z., Guan, X., *et al.*: 'User authentication through mouse dynamics', *IEEE Trans. Inf. Forensics Sec.*, 2013, **8**, (1), pp. 16–30
- [6] Shen, C., Cai, Z., Guan, X., *et al.*: 'Performance evaluation of anomaly-detection algorithms for mouse dynamics', *Comput. Secur.*, 2014, **45**, pp. 156–171
- [7] Fulop, A., Kovacs, L., Kurics, T., *et al.*: 'Balabit mouse dynamics challenge data set'. 2016. Available at <https://github.com/balabit/Mouse-Dynamics-Challenge>, accessed 26 January 2019
- [8] Gamboa, H., Fred, A.: 'A behavioral biometric system based on human-computer interaction'. Proc. SPIE 5404, Biometric Technology for Human Identification, 25 August 2004, 2004, vol. 5404, pp. 381–392
- [9] Nakkabi, Y., Traore, I., Ahmed, A.A.E.: 'Improving mouse dynamics biometric performance using variance reduction via extractors with separate features', *Trans. Syst. Man Cybern. A*, 2010, **40**, (6), pp. 1345–1353
- [10] Ahmed, A.A.E., Traore, I.: 'Dynamic sample size detection in continuous authentication using sequential sampling'. Proc. 27th Annual Computer Security Applications Conf. (ACSAC'11), New York, NY, USA, ACM, 2011, pp. 169–176
- [11] Zheng, N., Paloski, A., Wang, H.: 'An efficient user verification system using angle-based mouse movement biometrics', *ACM Trans. Inf. Syst. Secur.*, 2016, **18**, (3), pp. 11:1–11:27
- [12] Zheng, N., Paloski, A., Wang, H.: 'An efficient user verification system via mouse movements'. Proc. 18th ACM Conf. on Computer and Communications Security (CCS'11), New York, NY, USA, ACM, 2011, pp. 139–150
- [13] Feher, C., Elovici, Y., Moskovitch, R., *et al.*: 'User identity verification via mouse dynamics', *Inf. Sci.*, 2012, **201**, pp. 19–36
- [14] Chuda, D., Kratky, P.: 'Usage of computer mouse characteristics for identification in web browsing'. Proc. 15th Int. Conf. on Computer Systems and Technologies (CompSysTech'14), New York, NY, USA, ACM, 2014, pp. 218–225
- [15] Chuda, D., Kratky, P., Tvarozek, J.: 'Mouse clicks can recognize web page visitors!'. Proc. 24th Int. Conf. on World Wide Web (WWW'15 Companion), New York, NY, USA, ACM, 2015, pp. 21–22
- [16] Hinbarji, Z., Albatal, R., Gurrin, C.: 'Dynamic user authentication based on mouse movements curves'. MultiMedia Modeling, 2015, pp. 111–122
- [17] Carneiro, D., Castillo, J.C., Novais, P., *et al.*: 'Multimodal behavioral analysis for non-invasive stress detection', *Expert Syst. Appl.*, 2012, **39**, (18), pp. 13376–13389
- [18] Carneiro, D., Novais, P., Pego, J.M., *et al.*: 'Using mouse dynamics to assess stress during online exams', in Onieva, E., Santos, I., Osaba, E., *et al.* (Eds.): 'Hybrid artificial intelligent systems' (Springer International Publishing, Cham, 2015), pp. 345–356
- [19] Guo, Q., Agichtein, E.: 'Towards predicting web searcher gaze position from mouse movements'. CHI'10 Extended Abstracts on Human Factors in Computing Systems, 2010, pp. 3601–3606
- [20] Antal, M., Szabo, L.Z.: 'An evaluation of one-class and two-class classification algorithms for keystroke dynamics authentication on mobile devices'. 2015 20th Int. Conf. on Control Systems and Computer Science, 2015, pp. 343–350
- [21] Hall, M., Frank, E., Holmes, G., *et al.*: 'The weka data mining software: an update', *SIGKDD Explor. Newsl.*, 2009, **11**, (1), pp. 10–18
- [22] Breiman, L.: 'Random forests', *Mach. Learn.*, 2001, **45**, (1), pp. 5–32
- [23] Fawcett, T.: 'An introduction to ROC analysis', *Pattern Recognit. Lett.*, 2006, **27**, (8), pp. 861–874
- [24] Mondal, S., Bours, P.: 'Performance evaluation of continuous authentication systems', *IET Biometrics*, 2015, **4**, (4), pp. 220–226